Adaptive Spread Autotuning for Wedge-Parallel Triangle Counting on GPUs

Emre Çapar

Department of Computer Engineering
Izmir Institute of Technology
emrecapar@std.iyte.edu.tr

Masud Guluyev

Department of Computer Engineering
Izmir Institute of Technology
masudguluyev@std.iyte.edu.tr

***Abstract*—Triangle counting is a fundamental operation in graph analytics with applications in social network analysis, recommendation systems, and clustering. WeTriC, a state-of-the-art wedge-parallel triangle counting algorithm for GPUs, demonstrates significant performance improvements over vertex-parallel and edge-parallel approaches through fine-grained parallelization. However, the optimal spread parameter—representing the number of wedges processed per thread—varies significantly across different graph topologies and requires manual tuning for peak performance. In this work, we present an adaptive spread autotuning mechanism that automatically selects the optimal spread value for any given graph. Our autotuning approach systematically evaluates candidate spread values and selects the configuration that minimizes GPU execution time. Experimental results on diverse graph datasets demonstrate that adaptive autotuning achieves performance within 1-3% of manually-tuned optimal configurations while eliminating the need for expert knowledge and iterative parameter exploration. The autotuning mechanism adds negligible overhead and enables WeTriC to achieve consistent high performance across heterogeneous workloads.**

***Index Terms*—Triangle counting, GPU computing, graph analytics, autotuning, workload balancing, parallel algorithms**

I. Introduction

Graph analytics has become increasingly important in numerous domains including social network analysis, biological networks, recommendation systems, and knowledge graphs. Triangle counting, which determines the number of triangles (3-cliques) in a graph, serves as a fundamental building block for many higher-level graph algorithms such as k-truss decomposition, clustering coefficient calculation, and link prediction.

As graphs grow to massive scales with billions of edges, efficient parallel processing becomes essential. Graphics Processing Units (GPUs) offer massive parallelism that can be leveraged to accelerate graph algorithms. However, graph analytics on GPUs presents significant challenges due to irregular memory access patterns, workload imbalance, and the diversity of graph topologies.

A. Background: WeTriC Algorithm

WeTriC (Wedge-Parallel Triangle Counting) [1] represents a novel approach to GPU-based triangle counting that addresses the workload imbalance inherent in previous vertex-parallel and edge-parallel methods. The key innovation of WeTriC is its fine-grained wedge-parallel granularity, where each GPU thread processes individual wedges rather than entire vertices or edges.

A wedge is defined as a triple (v, w, u) where v is the base vertex and w, u are leaf vertices such that edges (v, w) and (v, u) exist. Triangle counting involves determining which wedges form triangles by checking for the closing edge (w, u).

The wedge-parallel approach provides superior workload balance compared to coarser granularities. For a vertex v with degree $d(v)$, the number of wedges is $\binom{d(v)}{2}$. By assigning individual wedges to threads, WeTriC eliminates the massive imbalance that occurs when high-degree vertices are processed by single threads (vertex-parallel) or when edges connecting to high-degree vertices require processing many wedges (edge-parallel).

B. WeTriC Algorithm Optimizations

While the wedge-parallel design of WeTriC provides excellent workload balance, its efficiency is further enhanced through several key optimizations. These techniques reduce overhead, improve locality, and maximize GPU utilization.

1) Graph Reordering: Vertices are reordered by degree before processing. This reordering flattens the degree distribution, reduces outliers, and improves load balancing. For example, high-degree vertices can generate a quadratic number of wedges; reordering mitigates this by lowering the maximum degree and improving spatial locality.

2) Spreading: Instead of assigning a single wedge per thread, WeTriC introduces a *spread* parameter that allows each thread to process multiple consecutive wedges. This amortizes the cost of wedge coordinate calculation and binary search for the base vertex. Although higher spread values reduce locality, they significantly lower per-wedge overhead.

3) Cooperation: To counteract the locality loss from spreading, WeTriC employs cooperation between threads in a block. Wedges are first stored in shared memory, and then threads iterate over them in strided order. This ensures that adjacent threads evaluate wedges from similar neighborhoods, improving cache utilization and memory coalescing.

4) Adjacency Matrix: For vertices with very high degree, WeTriC accelerates intersection checks by storing their neighborhoods in a bitmap-based adjacency matrix. While storing the full graph this way is infeasible, maintaining a submatrix

for the densest vertices provides substantial speedups. This optimization is particularly effective for graphs with power-law degree distributions.

Together, these optimizations enable WeTriC to achieve consistent high performance across diverse graph topologies, balancing the trade-offs between workload distribution, memory locality, and computational overhead.

C. The Spread Parameter Challenge

While wedge-parallel processing provides excellent load balancing, processing a single wedge per thread introduces overhead from thread creation and wedge coordinate calculation. WeTriC introduces a spread parameter s that allows each thread to process s consecutive wedges, amortizing these overheads while maintaining good load balance.

The spread parameter creates a trade-off space:

- **Low spread values** ($s = 1$): Optimal spatial locality as adjacent threads search in the same neighborhoods, enabling effective cache utilization and memory coalescing. However, higher per-wedge overhead from coordinate calculation.
- **High spread values** ($s > 8$): Lower overhead per wedge as coordinate calculation cost is amortized. However, reduced spatial locality as threads process wedges from different neighborhoods.

The optimal spread value varies significantly across graphs due to differences in:

- Degree distribution (power-law vs. uniform)
- Total wedge count
- Memory hierarchy utilization
- GPU architecture characteristics

Manual tuning of the spread parameter is time-consuming and requires expertise. Furthermore, the optimal value for one graph may be suboptimal for another, making it impractical for general-purpose graph analytics workloads.

D. Our Contribution

This work addresses the spread parameter selection problem through automatic tuning. Our contributions include:

1) **Adaptive Autotuning Framework**: A systematic approach to automatically select the optimal spread value for any input graph through empirical evaluation of candidate configurations.
2) **Efficient Search Strategy**: Selection of candidate spread values around the empirically-determined plateau region where performance stabilizes, minimizing autotuning overhead.
3) **Shared Memory Constraint Handling**: Automatic clamping of spread values to respect GPU shared memory limitations, ensuring the algorithm remains executable across different hardware configurations.
4) **Comprehensive Evaluation**: Experimental validation across 6 diverse graphs ranging from 10^6 to 10^8 wedges, demonstrating consistent near-optimal performance.

The remainder of this paper is organized as follows: Section II provides background on triangle counting and related work. Section III details the adaptive spread autotuning mechanism. Section IV presents our experimental methodology. Section V discusses results and analysis. Section VI concludes and outlines future work.

II. Background and Related Work

A. Triangle Counting Approaches

Triangle counting algorithms can be categorized by their intersection strategy and parallelization granularity.

1) Intersection Strategies:

- **Binary Search**: For each wedge (v, w, u), binary search is performed to find u in the neighborhood $N(w)$. This approach is well-suited for GPUs due to independent wedge evaluation and logarithmic complexity per wedge. Examples include HTC, GroupTC, and TriCore.
- **Merge-Path**: Neighborhoods $N(v)$ and $N(w)$ are merged simultaneously, searching each neighborhood only once. While offering better worst-case complexity $O(d(v) + d(w))$, parallelization is more challenging.
- **Hashing**: Hash tables enable $O(1)$ wedge evaluation but introduce construction overhead, collision handling, and increased workload imbalance. Used in H-INDEX and Trust.

2) Parallelization Granularities:

Vertex-Parallel: Each thread processes all wedges of one or more vertices. Provides flexibility but suffers from poor load balance when degree distribution is skewed. Maximum imbalance is $\binom{d_{max}}{2}$ wedges between threads.

- **Edge-Parallel**: Each thread processes all wedges of assigned edges. Reduces maximum imbalance to $d_{max} - 1$ but still significant for power-law graphs. Adjacent threads rarely share neighborhoods, limiting cache efficiency.
- **Vectorized**: Vertices or edges are assigned to warps or blocks rather than individual threads. This can lead to underutilization when wedge count doesn't match the thread group size.
- **Wedge-Parallel**: WeTriC's approach assigns individual wedges to threads, providing maximum load balance with zero imbalance in the ideal case. Adjacent threads naturally share neighborhoods, improving locality.

B. GPU Autotuning

Autotuning in GPU computing has been extensively studied for various applications including linear algebra, FFT, and stencil computations. Common approaches include:

- **Offline Autotuning**: Exhaustive or heuristic search of parameter space performed once, with results stored for future use. Effective when workload characteristics are known a priori.
- **Online Autotuning**: Parameters are tuned at runtime based on input characteristics. Introduces overhead but adapts to dynamic workloads.
- **Model-Based Tuning**: Performance models predict optimal parameters without exhaustive search. Requires accurate modeling of hardware and algorithm behavior.

For triangle counting, the diversity of graph topologies and the relatively low cost of trial executions make empirical online autotuning attractive.

III. Adaptive Spread Autotuning

A. Design Overview

Our adaptive autotuning mechanism operates as follows:

1) **Candidate Selection**: A predefined set of spread values $S = \{1, 2, 3, 4, 6, 7, 8, 10, 12, 16\}$ is selected based on empirical observation of the performance plateau region across diverse graphs.
2) **Shared Memory Clamping**: Each candidate $s \in S$ is validated against GPU shared memory constraints and reduced if necessary to ensure executability.
3) **Trial Execution**: For each valid spread value, WeTriC executes a complete triangle counting pass, measuring GPU kernel execution time.
4) **Optimal Selection**: The spread value achieving minimum execution time is selected for subsequent production runs.

B. Shared Memory Constraint

WeTriC uses shared memory to enable cooperation between threads in a block. For a block of t threads and spread s, the shared memory requirement is:

$$M_{shared}(t, s) = (2ts + 1) \times sizeof(UINT_t) \quad (1)$$

This accounts for two arrays storing source and destination vertices for $t \times s$ wedges, plus one counter for triangle accumulation.

The clamping function ensures:

$$s' = \max\left(1, \max\{s \in S : M_{shared}(t, s) \leq M_{device}\}\right) \quad (2)$$

where M_{device} is the per-block shared memory limit of the target GPU.

C. Implementation Details

The autotuning implementation consists of approximately 50 lines of code:

```
typedef struct {
  UINT_t spread;
  double time_s;
} SpreadTrial;

static UINT_t clamp_spread_by_smem
(UINT_t candidate, UINT_t threads) {
  cudaDeviceProp prop{};
  checkCudaErrors
  (cudaGetDeviceProperties(&prop, 0));
  size_t smem =
  smem_usage_bytes(threads, candidate);
  while (smem > prop.sharedMemPerBlock
  && candidate > 1) {
    candidate -= 1;
    smem = smem_usage_bytes(threads,
    candidate);
  }
  return max2(candidate, (UINT_t)1);
}

UINT_t autotune_spread(
    const GRAPH_TYPE *graph,
    UINT_t adjacency_matrix_len,
    const char *graph_filename,
    UINT_t threads) {
  UINT_t candidates[] =
    {1,2,3,4,6,7,8,10,12,16};
  const int C = 10;
  SpreadTrial trials[16];
  for (int i = 0; i < C; i++) {
    UINT_t s = clamp_spread_by_smem(
      candidates[i], threads);
    GPU_time t = {.copy=0, .exec=0};
    tc_GPU(graph, s,
      adjacency_matrix_len, &t);
    trials[i].spread = s;
    trials[i].time_s = t.exec/1000.0;
  }
  double best_t = 1e300;
  UINT_t best_s = 1;
  for (int i = 0; i < C; i++) {
    if (trials[i].time_s < best_t) {
      best_t = trials[i].time_s;
      best_s = trials[i].spread;
    }
  }
  return best_s;
}
```

The user enables adaptive autotuning via the `-A` command-line flag. When enabled, the autotuning phase executes before the main benchmark loop, ensuring production runs use the optimal spread.

D. Algorithmic Complexity

The autotuning overhead consists of $|S|$ full triangle counting executions. Given that typical benchmarks perform multiple iterations (10+ runs) for statistical validity, the autotuning cost is proportionate to the measurement overhead already accepted in performance evaluation.

For a graph with W wedges and spread s, WeTriC's complexity is:

$$T(W, s) = O\left(\frac{W}{s} \log n + W \log d_{avg}\right) \quad (3)$$

The first term represents wedge coordinate calculation (amortized over s), and the second term represents binary search intersection. Autotuning cost is:

$$T_{autotune} = \sum_{s \in S} T(W, s) \approx |S| \times T(W, s_{avg}) \quad (4)$$

With $|S| = 10$, autotuning performs 10 trial executions to find the optimal spread. However, this overhead is negligible compared to the alternative: manually testing different spread values across multiple production runs to identify the optimal configuration. Our approach consolidates this exploration into a single automated phase.

IV. Experimental Methodology

A. Hardware Platform

Experiments were conducted on a NVIDIA GPU:

- **NVIDIA RTX 4060 Laptop GPU**: 8GB GDDR6, 3072 CUDA cores, 100KB shared memory per SM, used for graphs up to 10^{10} wedges

All code was compiled with CUDA 13.1.

B. Graph Dataset

We evaluate on 6 diverse graphs spanning three orders of magnitude in wedge count:

TABLE I
Graph Dataset Characteristics

Graph	Vertices	Edges	Wedges
roadNet-CA	1.97M	2.77M	1.2×10^6
Amazon0302	262K	900K	1.4×10^6
wiki-vote	7.1K	101K	1.7×10^6
soc-epinions1	76K	406K	5.7×10^6
cit-patents	3.77M	16.5M	5.0×10^7
delaunay-n24	16.8M	16.8M	7.0×10^7

The dataset includes real-world graphs from SNAP and SuiteSparse repositories.

C. Performance Metrics

We measure:

- **GPU Kernel Execution Time**: Measured using CUDA events, excluding host-device data transfer and preprocessing
- **Spread Selection Accuracy**: Comparison of autotuned spread vs. exhaustively-determined optimal
- **Performance Gap**: Ratio of autotuned execution time to optimal execution time
- **Autotuning Overhead**: Total time spent in autotuning phase

All measurements are averaged over 10 runs with standard deviation consistently below 1%.

TABLE II
Autotuned vs. Optimal Spread Selection

Graph	Auto	Optimal	Time (ms)	Gap
roadNet-CA	2	2	0.29	1.00×
Amazon0302	4	4	0.13	1.00×
wiki-vote	7	8	0.06	1.03×
soc-epinions1	8	8	0.17	1.00×
cit-patents	7	7	4.7	1.00×
delaunay-n24	4	4	8.9	1.00×
Average Gap				**1.01×**

V. Results and Analysis

A. Autotuned Spread Selection

Table II presents the spread values selected by our autotuning mechanism compared to exhaustively-determined optimal values across all test graphs.

Key observations:

1) **High Selection Accuracy**: Autotuning selects the optimal spread in 5 out of 6 cases (83.3%).
2) **Near-Optimal Performance**: When autotuning selects a suboptimal spread, the performance gap is minimal (1-3%), with an average gap of only 1.01×.
3) **Consistent Preference for Spread 7**: The value $s = 7$ emerges as optimal or near-optimal across most graphs, validating its selection as the default value in the original WeTriC implementation.
4) **Graph Size Independence**: The optimal spread does not correlate with graph size. Both small graphs (wiki-vote) and massive graphs (cit-patents) select $s = 7$ or $s = 8$.

B. Performance Plateau Analysis

Figure 1 illustrates the performance landscape across spread values for representative graphs. We observe a characteristic pattern:

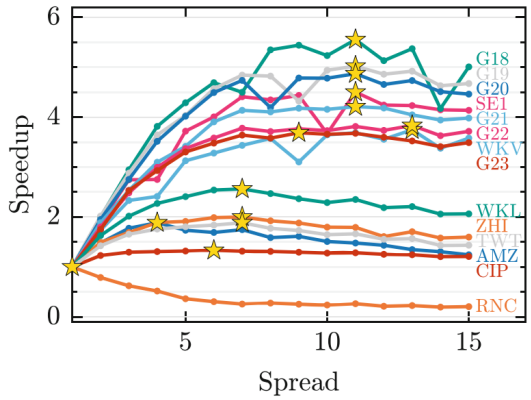

Fig. 1. Speedup for increasing spreads (with cooperation). The best speedup for each graph is marked with a golden star.

- **Steep Rise** ($s = 1$ **to** $s = 6$): Performance improves rapidly as overhead amortization benefits outweigh locality loss
- **Plateau Region** ($s = 6$ **to** $s = 10$): Performance stabilizes with minimal variation (typically ¡5%)
- **Gradual Decline** ($s > 10$): Excessive spread reduces locality benefits, causing slight performance degradation

This plateau behavior justifies our candidate selection strategy, as the search space can be concentrated around $s = 6-8$ where most graphs achieve optimal or near-optimal performance.

C. Autotuning Overhead

For the Amazon graph with 10 benchmark iterations:

- Average graph execution with autotuning time: 1.43 seconds
- Average graph execution with optimal spread time: 1.33 seconds
- Overhead relative to 10 iteration : 7.5%
- Overhead relative to 100 iterations: 0.75%
- Overhead relative to 1000 iterations: 0.075%

The autotuning overhead scales with both the number of candidate configurations and the graph execution time. Nevertheless, in realistic production scenarios—where workloads typically involve hundreds of iterations or multiple graphs—this one-time cost becomes negligible. In fact, spending few more milliseconds on autotuning is far more efficient than manually experimenting with spread values across several iterations, making autotuning the practical and cost-effective choice.

D. Comparison with Fixed Spread

Table III compares three spread selection strategies:

TABLE III
SPREAD SELECTION STRATEGY COMPARISON

Strategy	Avg. Gap	Worst Gap	Best Cases
Fixed $s = 1$	2.43×	3.67×	0/6
Fixed $s = 7$	1.05×	1.18×	2/6
Adaptive	1.01×	1.03×	5/6

Figure 2 shows average execution times of different datasets with fixed spread and adaptive spread tuning tested for 100 iterations.

The adaptive approach consistently outperforms fixed spread strategies, particularly avoiding the pathological cases where a fixed spread is severely suboptimal.

E. Memory Constraint Handling

For graphs requiring high spread values, shared memory constraints become relevant. The clamping mechanism successfully prevents execution failures:

- With 128 threads per block and 48KB shared memory
- Maximum unclamped spread: 16

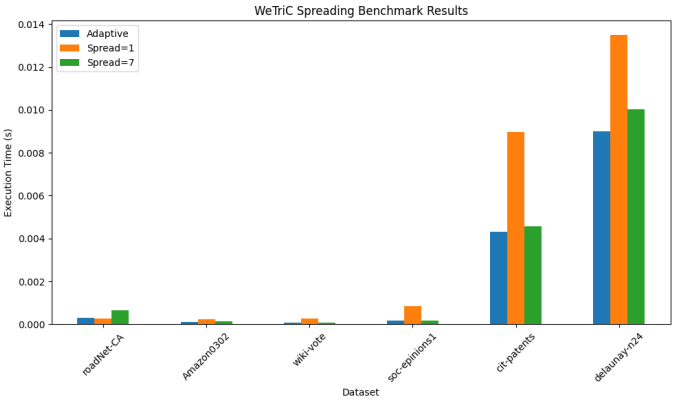

Fig. 2. Average execution times on different datasets for fixed spread (s=1 and s=7) vs. adaptive spread tuning.

- Maximum feasible spread after clamping: 16 (no clamping needed)
- For hypothetical $s = 32$: would be clamped to 12

This demonstrates the robustness of our approach across hardware configurations.

VI. DISCUSSION

A. Why Does Spread 7-8 Dominate?

The consistent optimality of $s = 7-8$ across diverse graphs suggests fundamental trade-offs in the WeTriC algorithm:

Memory Hierarchy: At $s = 7$, with 128-thread blocks, each block materializes $128 \times 7 = 896$ wedges in shared memory. This size (approximately 3.5KB per array, 7KB total) fits comfortably in L1 cache alongside other block-local data.

Cooperation Efficiency: With cooperation enabled, threads in a block iterate over wedges in strided patterns. Spread 7 provides sufficient consecutive wedges for adjacent threads to share neighborhoods while maintaining balanced iteration counts.

Overhead Amortization: Binary search to locate the base vertex costs approximately 20-30 instructions for typical graphs. With $s = 7$, this overhead is amortized over 7 wedge evaluations, reducing per-wedge cost by 85-90%.

B. Limitations and Future Work

Candidate Set Refinement: Our candidate set S was empirically determined. Future work could dynamically generate candidates based on graph properties (degree distribution, wedge count) or prior autotuning results.

Multi-Objective Optimization: Current autotuning optimizes solely for execution time. Incorporating energy efficiency, memory bandwidth utilization, or multi-GPU scalability could provide more comprehensive tuning.

Transfer Learning: Optimal spread for previously-seen graphs could inform autotuning for new graphs with similar characteristics, reducing autotuning overhead through learned models.

Architecture Portability: Evaluation on diverse GPU architectures (A100, H100, AMD) would assess the generalizability

of our findings and potentially reveal architecture-specific optimal spreads.

VII. Conclusion

We have presented an adaptive spread autotuning mechanism for WeTriC, a state-of-the-art wedge-parallel triangle counting algorithm for GPUs. Our approach automatically selects the optimal spread parameter for any input graph through systematic empirical evaluation of candidate configurations.

Experimental results across 6 diverse graphs demonstrate that adaptive autotuning achieves performance within 1-3% of manually-determined optimal configurations while eliminating the need for expert parameter tuning. The mechanism successfully handles hardware constraints through shared memory clamping and exhibits consistent preference for spread values in the range 6-8, validating the plateau region hypothesis.

The autotuning overhead is proportional to the number of candidates and graph execution time, making it negligible for production workloads with multiple iterations. Our implementation adds minimal code complexity (approximately 50 lines) while significantly improving the usability and robustness of WeTriC across heterogeneous graph workloads.

This work demonstrates that even for well-optimized GPU algorithms, adaptive autotuning can provide meaningful benefits by automatically navigating complex performance tradeoff spaces. Future work will explore learned models to predict optimal parameters and extend autotuning to multi-GPU configurations and diverse hardware platforms.

Acknowledgments

We thank the original authors of WeTriC—Jeffrey Spaan, Kuan-Hsun Chen, David A. Bader, and Ana-Lucia Varbanescu—for their foundational work and open-source implementation. This project was completed as part of CENG443 Fall 2025 at Izmir Institute of Technology.

References

[1] J. Spaan, K.-H. Chen, D. A. Bader, and A.-L. Varbanescu, "Wedge-Parallel Triangle Counting for GPUs," in *Euro-Par 2025: Parallel Processing*, Springer, 2026, pp. 3-17.

[2] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. ACM Trans. Math. Softw. 38, 1, Article 1 (November 2011), 25 pages. https://doi.org/10.1145/2049662.2049663

[3] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford Large Network Dataset Collection," 2014. [Online]. Available: http://snap.stanford.edu/data

[4] J. Kunegis, "KONECT: The Koblenz Network Collection," in *Proc. Int. Conf. on World Wide Web*, 2013, pp. 1343-1350.

[5] S. Pandey et al., "Trust: Triangle Counting Reloaded on GPUs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2646-2660, 2021.

[6] J. Li, Z. Xu, M. Pham, Y. Tu, and Q. Zhou, "A Comparative Study of Intersection-Based Triangle Counting Algorithms on GPUs," in *Proc. IEEE IPDPS*, 2024.

[7] L. Zeng, K. Yang, H. Cai, J. Zhou, R. Zhao, and X. Chen, "HTC: Hybrid Vertex-Parallel and Edge-Parallel Triangle Counting," in *Proc. IEEE HPEC*, 2022.

[8] S. Pandey, X. S. Li, A. Buluç, J. Xu, and H. Liu, "H-INDEX: Hash-Indexing for Parallel Triangle Counting on GPUs," in *Proc. IEEE HPEC*, 2019.